

Loss Functions: Surrogates, Calibration, and What Each Loss Remembers

Yuval Lavie

August 10, 2026

Every model in this repository is “a loss plus an optimizer.” This page is the loss catalogue: what each one is, where it comes from, what its population minimizer *remembers* about the data, and how it treats outliers. `losses_demo.py` verifies the quantitative claims.

1 The loss we mean, and the losses we use

For classification the loss we *care about* is the 0-1 loss $\mathbf{1}[h(x) \neq y]$ — the learnability note’s object. We rarely optimize it directly: it is discontinuous, non-convex, and ERM under it is NP-hard for most classes. Instead we minimize a *surrogate*: a convex loss whose minimization is tractable and whose minimizer still classifies correctly. The whole catalogue below is organized by this divide — the true loss, and the roofs we build over it.

2 Margin losses for binary classification

With $y \in \{-1, +1\}$ and a real-valued score $f(x)$, every loss below depends only on the *margin* $m = y f(x)$ (positive iff correct, large iff confidently correct):

$$\ell_{01} = \mathbf{1}[m \leq 0], \quad \ell_{\text{hinge}} = \max(0, 1 - m), \quad \ell_{\log} = \log(1 + e^{-m}), \quad \ell_{\text{exp}} = e^{-m}, \quad (1)$$

plus the squared hinge $\max(0, 1 - m)^2$ and the squared loss $(1 - m)^2$. Provenance: hinge is the SVM’s slack (svm note); the logistic loss *is* the Bernoulli NLL in ± 1 convention ($\log(1 + e^{-yf}) = -\log \sigma(yf)$ — cross-entropy, the MLE notes); the exponential loss is AdaBoost’s; the perceptron loss $\max(0, -m)$ drives the perceptron update. All the convex members dominate the 0-1 loss pointwise ($\ell \geq \ell_{01}$, asserted on a grid in the script), so small surrogate risk forces small classification risk: $\Pr(\text{error}) \leq \mathbb{E}[\ell(m)]$.

3 Calibration: what the minimizer remembers

The deeper question about a surrogate: at a point x with $p = \Pr(Y = +1|x)$, what score does minimizing the *population* risk $p\ell(f) + (1 - p)\ell(-f)$ produce?

Proposition 1 (Population minimizers).

$$f_{\log}^* = \log \frac{p}{1 - p}, \quad f_{\text{sq}}^* = 2p - 1, \quad f_{\text{hinge}}^* = \text{sign}(2p - 1), \quad f_{\text{exp}}^* = \frac{1}{2} \log \frac{p}{1 - p}. \quad (2)$$

(Each by setting the derivative of $p\ell(f) + (1-p)\ell(-f)$ to zero; the script confirms all four numerically.) Read the hierarchy:

- **Logistic:** $\sigma(f^*) = p$ — the full conditional probability survives. Minimizing log-loss *is* estimating $p(x)$; this is why logistic regression is calibrated and why cross-entropy owns probabilistic ML.
- **Squared:** p is recoverable ($p = (f^* + 1)/2$), but the loss keeps penalizing beyond $m > 1$ — confidently correct points are punished, dragging boundaries (multi-output note’s one-hot trick defect).
- **Hinge:** only $\text{sign}(2p - 1)$ survives — the *class*, not the probability. SVM scores are not calibrated; in exchange, the flat region ($\ell = 0$ for $m \geq 1$) ignores confident points entirely, producing the support-vector sparsity.

All of them are *classification-calibrated* ($\text{sign } f^* = \text{sign}(p - \frac{1}{2})$, the Bayes rule): minimizing any of these surrogates over rich enough classes drives the 0-1 risk to the Bayes risk (Bartlett–Jordan–McAuliffe). You choose among them by what you need to *keep* — probabilities, sparsity, or simplicity — not by correctness.

4 Regression losses: location and robustness

For residual $r = y - f(x)$:

$$\ell_{\text{sq}} = \frac{1}{2}r^2, \quad \ell_{\text{abs}} = |r|, \quad \ell_{\text{Huber},\delta} = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta(|r| - \frac{\delta}{2}) & |r| > \delta, \end{cases} \quad (3)$$

plus the ε -insensitive $\max(0, |r| - \varepsilon)$ of SVR. Their population minimizers are the classical location statistics: squared $\rightarrow$ conditional *mean* (the Gaussian MLE note), absolute $\rightarrow$ conditional *median*, Huber $\rightarrow$ in between. That single fact is the robustness story: a gross outlier contributes $O(r^2)$ to squared loss but only $O(|r|)$ to absolute and Huber — the script fits a constant to data with 10% gross outliers, and the squared fit is dragged to 9.5 while absolute and Huber stay at the bulk’s 5.0. Huber keeps the squared loss’s smoothness and statistical efficiency near zero while capping the influence of the tails — the standard compromise.

5 Steepness is a personality trait

For $m < 0$ (mistakes), compare growth: hinge is linear, logistic is asymptotically linear, exponential is *exponential*. The script prints the penalty a badly mislabeled point pays at score $|f| = 10$: hinge 11, exponential $e^{10} \approx 22000$ — AdaBoost’s exquisite outlier sensitivity in one number. Flat-side behavior matters too: hinge is exactly 0 for $m \geq 1$ (satisfied points are ignored: sparsity, PA’s “passive”), while logistic never reaches zero (every point keeps voting: no support-vector structure, but calibrated probabilities). Steep left side = aggressive on errors; flat right side = indifferent to safe points; every loss in (1) is a different setting of those two dials.

6 Choosing, in one table

loss	convex	minimizer remembers	outlier response	used by
0-1	no	the class	indifferent	the target itself
hinge	yes	the class	linear	SVM, PA
squared hinge	yes	class (+ margin)	quadratic	smooth SVMs
logistic / CE	yes	the probability	$\sim$ linear	GLMs, deep nets
exponential	yes	$\frac{1}{2}$ log-odds	exponential	AdaBoost
squared (cls.)	yes	$2p - 1$	quadratic (both sides!)	one-hot LS
squared (reg.)	yes	mean	quadratic	OLS, Gaussian MLE
absolute	yes	median	linear	robust regression
Huber	yes	mean $\leftrightarrow$ median	quad. then linear	robust default

One sentence to keep: *a loss function is a statement about which mistakes you fear and what information you want the minimizer to retain* — probabilities (logistic), the decision alone (hinge), or a robust location (absolute/Huber); everything else in this repository is the machinery for descending it.